

Optimizing the Computer Science Catalog

Paula Lozano Gonzalo

December 6th 2024

Contents

1	Introduction	2
2	PEAS Assessment	2
3	Search Algorithm	2
4	Agent Implementations	2
5	Problem Instances and Results	3
5.1	Catalog Search Results	3
5.2	Agent Results	3
6	Conclusion	4

1 Introduction

The objective of this project is to optimize the Computer Science course catalog to enhance student success. Currently, the primary metric for evaluation is the graduation rate, with future plans to incorporate the balanced distribution of challenging courses between semesters. The environment is modeled as a deterministic and fully observable system, assuming consistent degree requirements for the 2024–2025 academic year.

2 PEAS Assessment

Performance Measure:

- **Graduation Rate:** Percentage of students who graduated within a specified time frame. In the actual environment this would be 5 to 6 semesters because general are not included.

Environment:

- **Observability:** Fully observable. All required courses, their prerequisites, and the necessary credits are known ahead of time. This ensures that the agent has complete information to make informed decisions when taking classes.
- **Uncertainty:** Deterministic. For simulation, it is assumed that once a student takes a class, they pass it, reducing the number of required classes for graduation.
- **Stability:** Static. The degree requirements remain consistent over time, specifically for the Computer Science catalog for the 2024–2025 academic year. This static nature ensures stability in the simulation and optimization process.
- **Granularity:** Discrete. All values in the environment, such as course offerings, credits, and prerequisites, are represented as discrete entities. This granularity simplifies the modeling and simulation process.
- **Participants:** Single agent. The agent operates independently to generate and evaluate course catalog configurations without any competition or collaboration with other agents.

Actuators:

- Generating students to take classes based on semester offerings and prerequisites and/or other parameters.

Sensors:

- Current course availability with additional information such as a catalog dictionary detailing course 'ID', 'Credit Hours', 'Prerequisites', and 'Designation'. It also accounts for the maximum credits per semester and tracks progress toward meeting graduation requirements.

3 Search Algorithm

The **random restart hill climbing algorithm** was employed to identify catalog configurations that maximize utility. Utility was defined as the average number of semesters required for students to graduate. Increasing the number of random restarts and the number of simulated students improved accuracy but came at the cost of increased runtime. The utility function was intentionally kept simple to introduce more randomness into the environment, resulting in greater variability in measurements.

4 Agent Implementations

Random Agent: Selects courses randomly from those available each semester in the observation received.

Requirements Agent: Prioritizes courses based on graduation requirements, avoiding redundant classes once the requirements are met.

5 Problem Instances and Results

Experiments varied the number of students (100, 1,000, 10,000) and credits per semester (12, 15, 18, 20), as well as the number of random restarts (5, 10, 50).

5.1 Catalog Search Results

Table 1: Utility and Runtime for Random Restart Hill Climbing

Restarts	Students	Utility (semesters)	Runtime (seconds)
5	100	9.63	5.80
10	100	9.68	11.15
50	100	9.69	56.30
5	1,000	9.74	50.06
10	1,000	9.70	114.44
50	1,000	9.74	565.90
10	10,000	9.72	1,105.58

It is important to note that the utility of runs with different numbers of students cannot be directly compared. Since we are using a student simulator, a larger number of students leads to more accurate results. Additionally, the current utility function, which operates similarly to a random agent, does not accurately simulate real student behavior. A more effective approach would be to use something like the requirements agent, which would better reflect realistic student course selection patterns.

5.2 Agent Results

Note: the catalog used for all of these runs is the same for both agents, and it is the original catalog that Utah Tech offers right now.

Random Agent: Performance varied based on credits per semester.

Table 2: Random Agent Results (100 Students)

Credits	Avg Semesters	Min Semester	Max Semester	Students
12	15.00	12	20	100
15	12.00	10	17	100
18	11.00	8	14	100
20	10.00	7	14	100

As shown in Table 2, the results were more than double than what we expected. Since we don't have generals included in the catalog it only takes students up to 5 or 6 semester to graduate. However, in this agent the average graduation time was at least doubled.

Table 3: Random Agent Results (1,000 Students)

Credits	Avg Semesters	Min Semester	Max Semester	Students
12	14.00	10	21	1,000
15	12.00	9	18	1,000
18	11.00	7	16	1,000
20	10.00	8	15	1,000

Compared to Table 2, Table 3 shows similar results, but with more students generated for the simulation, the results are more accurate.

Requirements Agent: Delivered consistently lower average semesters.

Table 4: Requirements Agent Results (1,000 Students)

Credits	Avg Semesters	Min Semester	Max Semester	Students
12	8.00	7	10	100
15	6.00	6	8	100
18	5.00	5	6	100
20	5.00	5	6	100

Table 5: Requirements Agent Results (1,000 Students)

Credits	Avg Semesters	Min Semester	Max Semester	Students
12	7.00	7	10	1,000
15	6.00	6	8	1,000
18	5.00	5	6	1,000
20	5.00	5	6	1,000

Tables 4 and 5 show a significant improvement compared to the random agent seen earlier. The graduation expectancy is based on a limit of 15 credits per semester, which is why with 12 credits the average is 7 semesters, as students are not taking as many credits as they should. The subsequent results aligned with expectations, but upon reviewing the agent’s output, I noticed that sometimes too many challenging courses were taken simultaneously. I don’t believe this is realistic, which is why a future agent would incorporate weightings to better account for course difficulty and student capacity.

6 Conclusion

This project demonstrated the utility of random restart hill climbing and agent-based simulations in optimizing the Computer Science course catalog. The Requirements Agent consistently outperformed the Random Agent, emphasizing the importance of aligning course selection with graduation requirements. However, it also revealed issues, such as scheduling difficult courses in a single semester. Future improvements could involve adding constraints to balance the workload across semesters and account for courses that serve as prerequisites for multiple others.